C# Interview questions

1. **What is C#?**

C# (C-Sharp) is a modern, general-purpose, object-oriented programming language (OOP) developed by Microsoft. It's primarily used on the Windows .NET framework, but can also be used on open source platforms.

C# is used in many areas of software development, including: Web and desktop app development, Game development, Microsoft Visual Studio, and Paint.NET

C# can be used to build a variety of things, including:

Windows client apps, Libraries, Components, Services, APIs, iOS apps, Android apps, Interoperability tools, Gaming systems, and Games.

2. What are the Features of C#

C# is a general-purpose programming language with many features, including:

1. Object-oriented: C# is a fully object-oriented language, which makes it easier to develop and maintain code.
2. Type safe: C# code can only access memory locations that it has permission to execute.
3. Interoperability: C# allows you to take advantage of existing investments in unmanaged code.
4. String interpolation: Introduced in C# 6.0, this feature allows you to embed expressions and variables directly within string literals.
5. Scalable and updateable: C# is a popular language for creating robust applications that can scale and be updated.
6. Fast: C# can make coding faster than other programming languages.
7. Easy to learn: C# is designed to be easy to learn, especially for programmers familiar with languages like Java and C++.

3.What are the difference between java and C#

Feature	C#	Java
Operator Overloading	C# supports operator overloading for multiple operators.	Java does not support operator overloading.
Runtime Environment	C# supports CLR(Common Language Runtime).	Java supports JVM(Java Virtual Machine).
API Control	C# APIs are controlled by an open-source community.	Java APIs are also controlled by open community processes support.
Public Classes	In C#, there can be many public classes inside a source code.	In Java there can be only one public class inside a source code otherwise there will be a compilation error.
Checked Exceptions	C# does not support Unix-based checked exceptions. In some cases checked exceptions are very useful for the smooth execution of the program.	Java supports both checked and unchecked exceptions.
Platform Dependency	C# is cross-platform and runs on both Windows & Unix-based systems.	Java is a robust and platform-independent language. The platform independence of Java is through JVM.

4. Difference between C# and C++

Difference between C++ and C#(Sharp)

Below are some major differences between C++ and C#:

Feature	C++	C#
Memory Management	In C++ memory management is performed manually by the programmer. If a programmer creates an object then he is responsible to destroy that object after the completion of that object's task.	In C# memory management is performed automatically by the garbage collector. If the programmer creates an object and after the completion of that object's task the garbage collector will automatically delete that object.
Platform Dependency	C++ code can be run on any platform. C++ is used where the application needed to directly communicate with hardware.	C# code is windows specific. Although Microsoft is working to make it global but till now the major system does not provide support for C#.
Multiple Inheritance	C++ support multiple inheritance through classes. Means that a class can extend more than one class at a time.	C# does not support any multiple inheritances through classes.
Bound Checking	In C++ bound checking is not performed by compiler. By mistake, if the programmer tries to access invalid array index then it will give the wrong result but will not show any compilation error.	In C# bound checking in array is performed by compiler. By mistake, if the programmer tries to access an invalid array index then it will give compilation error.

Feature	C++	C#
Pointers	In C++ pointers can be used anywhere in the program.	In C# pointers can be used only in unsafe mode.
Language Type	C++ is a low level language.	C# is high level object oriented language.
Level of Difficulty	C++ includes very complex features.	C# is quite easy because it has the well-defined hierarchy of classes.
Application Types	C++ is typically used for console applications.	C# is used to develop mobile, windows, and console applications.
Compilation	C++ code gets converted into machine code directly after compilation.	C# code gets converted into intermediate language code after compilation.
Object Oriented	C++ is not a pure object-oriented programming language due to the primitive data types.	C# is a pure object-oriented programming language.
Access Specifiers	The access modifiers are public, private, protected. It does not contain internal & protected internal access modifiers.	In C# public, private, protected, internal & protected internal are used for access specifiers.

Feature	C++	C#
Test Variable	In switch statement, the test variable can not be a string.	In switch statement, the test variable can be a string.
Control statement	It does not contain such extra flow control statement.	In addition to for, while and do while; it has another flow control statement called for each.
Function Pointers	It does have the concept of function pointers.	C# does have the concept of function pointers, but they are implemented differently compared to languages like C++. In C#, delegates serve as function pointers.
Binaries	In C++ , size of binaries is low and lightweight.	In C# , size of binaries is high because of overhead libraries.
Garbage Collection	C++ do not support garbage collection.	Garbage collection is supported by C#
Types of Projects	It is mainly used for such projects that focus on accessing the hardware and better performance.	It is mainly used in modern application development.

5. Can multiple catch blocks be executed?

No, you cannot execute multiple catch blocks of the same type.

6. What is the difference between static, public, and void?

Public declared variables can be accessed from anywhere in the application. Static declared variables can be accessed globally without needing to create an instance of the class. Void is a type modifier which states the method and is used to specify the return type of a method in C#.

7. What are Jagged Arrays?

The Array which comprises elements of type array is called Jagged Array. The elements in Jagged Arrays can be of various dimensions and sizes.

8. What is the difference between out and ref parameters?

When an argument is passed as a ref, it must be initialized before it can be passed to the method. An out parameter, on the other hand, need not to be initialized before passing to a method.

9. What is the benefit of 'using' statement in C#?

The 'using' statement can be used in order to obtain a resource for processing before automatically disposing it when execution is completed.

10. Can "this" command be used within a static method?

No, we can't use "this" keyword inside a static method. "this" refers to current instance of the class. But if we define a method as static , class instance will not have access to it, only CLR executes that block of code. Hence we can't use "this" keyword inside static method.

11. Differentiate between Break and Continue Statement.

Continue statement - Used in jumping over a particular iteration and getting into the next iteration of the loop.

Break statement - Used to skip the next statements of the current iteration and come out of the loop.

12. List the different types of comments in C#.

The different types of comments in C# are:

XML comments

Example -

/// example of XML comment

Single Line comments

Example -

// example of single-line comment

Multi-line comments

Example -

/* example of an

multiline comment */

13. Explain the four steps involved in the C# code compilation.

Four steps of code compilation in C# include -

1. Source code compilation in managed code.
2. Newly created code is clubbed with assembly code.
3. The Common Language Runtime (CLR) is loaded.
4. Assembly execution is done through CLR.

14. Discuss the various methods to pass parameters in a method.

The various methods of passing parameters in a method include -

1. Output parameters: Lets the method return more than one value.
2. Value parameters: The formal value copies and stores the value of the actual argument, which enables the manipulation of the formal parameter without affecting the value of the actual parameter.
3. Reference parameters: The memory address of the actual parameter is stored in the formal argument, which means any change to the formal parameter would reflect on the actual argument too.

15. Name all the C# access modifiers.

The C# access modifiers are -

1. Private Access Modifier - A private attribute or method is one that can only be accessed from within the class.
2. Public Access Modifier - When an attribute or method is declared public, it can be accessed from anywhere in the code.
3. Internal Access Modifier - When a property or method is defined as internal, it can only be accessible from the current assembly point of that class.
4. Protected Access Modifier - When a user declares a method or attribute as protected, it can only be accessed by members of that class and those who inherit it.

16. Mention the important IDEs for C# development provided by Microsoft.

The following IDEs' are useful in C# development -

1. MonoDevelop
2. Visual Studio Code (VS Code)
3. Browxy
4. Visual Studio Express (VSE)
5. Visual Web Developer (VWD)

17. Why do we use C# language?

Below are the reasons why we use the C# language -

1. C# is a component-oriented language.
2. It is easy to pass parameters in the C# language.
3. The C# language can be compiled on many platforms.
4. The C# language follows a structured approach.
5. It is easy to learn and pick up.
6. The C# language produces really efficient and readable programmes.

18. What is meant by Unmanaged or Managed Code?

1. Managed code is the code which is managed by the CLR(Common Language Runtime) in .NET Framework. Whereas the Unmanaged code is the code which is directly executed by the operating system.
2. In simple terms, managed code is code that is executed by the CLR (Common Language Runtime). This means that every application code is totally dependent on the .NET platform and is regarded as overseen in light of it. Code executed by a runtime programme that is not part of the .NET platform is considered unmanaged code. Memory, security, and other activities related to execution will be handled by the application's runtime.

19. What is meant by a Partial Class?

1. Technically, a partial class is a single class that can be divided into different files, all within the same assembly. Due to assembly boundaries, you can't have a part of a class in one assembly and another part in a different assembly
2. A partial class effectively breaks a class's definition into various classes in the same or other source code files. A class definition can be written in numerous files, but it is compiled as a single class at runtime, and when a class is formed, all methods from all source files can be accessed using the same object. The keyword 'partial' denotes this.

20. What is the difference between read-only and constants?

During the time of compilation, constant variables are declared as well as initialized. It’s not possible to change this particular value later. On the other hand, read-only is used after a value is assigned at run time.

21. What are reference types and value types?

A value type holds a data value inside its memory space. Reference type, on the other hand, keeps the object's address where the value is stored. It is, essentially, a pointer to a different memory location.

22. What are User Control and Custom Control?

Custom Controls are produced as compiled code. These are easy to use and can be added to the toolbox. Developers can drag and drop these controls onto their web forms. User Controls are almost the same as ASP include files. They are also easy to create. User controls, however, can't be put in the toolbox. They also can't be dragged and dropped from it.

23. What are sealed classes in C#?

When a restriction needs to be placed on the class that needs to be inherited, sealed classes are created. In order to prevent any derivation from a class, a sealed modifier is used. Compile-time error occurs when a sealed class is forcefully specified as a base class.

24. Is it possible for a private virtual method to be overridden?

A private virtual method cannot be overridden as it can't be accessed outside the class.

25. Describe the accessibility modifier "protected internal".

Variables or methods that are Protected Internal can be accessed within the same assembly as well as from the classes which have been derived from the parent class.

26. What are the differences between System.String and System.Text.StringBuilder classes?

System.String is absolute. When a string variable's value is modified, a new memory is assigned to the new value. The previous memory allocation gets released.

System.StringBuilder, on the other hand, is designed so it can have a mutable string in which a plethora of operations can be performed without the need for allocation of a separate memory location for the string that has been modified.

27. What's the difference between the System.Array.CopyTo () and System.Array.Clone () ?

In the Clone () method, a new array object is created, with all the original Array elements using the CopyTo () method. Essentially, all the elements present in the existing array get copied into another existing array.

28. How can the Array elements be sorted in descending order?

You can use the Using Sort() methods and then Reverse() method.

29. What is the difference between Dispose() and Finalize()methods?

Dispose() is used when an object is required to release any unmanaged resources in it. Finalize(), on the other hand, doesn't assure the garbage collection of an object even though it is used for the same function.

30. What are circular references?

When two or more resources are dependent on each, it causes a lock condition, and the resources become unusable. This is called a circular reference.

31. What are generics in C# .NET?

In order to reduce code redundancy, raise type safety, and performance, generics can be used in order to make code classes that can be reused. Collection classes can be created using generics.

32. What is an object pool in .NET?

A container that has objects which are ready to be used is known as an object pool. It helps in tracking the object which is currently in use and the total number of objects present in the pool. This brings down the need for creating and re-creating objects.

33. List down the most commonly used types of exceptions in .NET

Commonly used types of exceptions in .NET are:

ArgumentException

ArithmeticException

DivideByZeroException

OverflowException

InvalidCastException

InvalidOperationException

NullReferenceException

OutOfMemoryException

StackOverflowException

34. What are Custom Exceptions?

In some cases, errors have to be handled according to user requirements. Custom exceptions are used in such cases.

35. What are delegates?

Delegates are essentially the same as function pointers in C++. The main and only difference between the two is delegates are type safe while function pointers are not. Delegates are essential because they allow for the creation of generic type-safe functions.

36. How do you inherit a class into another class in C#?

In C#, colon can be used as an inheritance operator. You need to place a colon and follow it with the class name.

37. Why can't the accessibility modifier be specified for methods within the interface?

In an interface, there are virtual methods which do not come with method definition. All the methods present are to be overridden in the derived class. This is the reason they are all public.

38. How can we set the class to be inherited, but prevent the method from being overridden?

To set the class to be inherited, it needs to be declared as public. The method needs to be sealed to prevent any overrides.

39. What happens if the method names in the inherited interfaces conflict?

A problem could arise when the methods from various interfaces expect different data. But when it comes to the compiler itself, there shouldn't be an issue.

40. What is the difference between a Struct and a Class?

Structs are essentially value-type variables, whereas classes would be reference types.

41. How to use nullable types in .Net?

When either normal values or a null value can be taken by value types, they are called nullable types.

42. How can we make an array with non-standard values?

An array with non-default values can be created using Enumerable.Repeat.

43. What is the difference between "is" and "as" operators in c#?

An "is" operator can be used to check an object's compatibility with respect to a given type, and the result is returned as a Boolean. An "as" operator can be used for casting an object to either a type or a class.

44. What is a multicast delegate?

Multicast delegate is when a single delegate comes with multiple handlers. Each handler is assigned to a method.

45. What are indexers in C# .NET?

In C#, indexers are called smart arrays. Indexers allow class instances to be indexed in the same way as arrays do.

46. What is the distinction between "throw" and "throw ex" in.NET?

“Throw” statement keeps the original error stack. But “throw ex” keeps the stack trace from their throw point.

47. What are C# attributes and its significance?

C# gives developers an option to define declarative tags on a few entities. For instance, class and method are known as attributes. The information related to the attribute can be retrieved during runtime by taking the help of Reflection

48. In C#, how do you implement the singleton design pattern?

In a singleton pattern, a class is allowed to have only one instance, and an access point is provided to it globally.

49. What's the distinction between directcast and ctype?

If an object is required to have the run-time type similar to a different object, then DirectCast is used to convert it. When the conversion is between the expression as well as the type, then Ctype is used.

50. Is C# code managed or unmanaged code?

C# is a managed code as the runtime of Common language can compile C# code to Intermediate language.

51. What is a Console application?

An application that is able to run in the command prompt window is called a console application.

52. What are namespaces in C#?

Namespaces allow you to keep one set of names that is different from others. A great advantage of namespace is that class names declared in one namespace don't clash with those declared in another namespace.

53. Difference between SortedList and SortedDictionary in C#.

SortedList is a collection of value pairs sorted by their keys. SortedDictionary is a collection to store the value pairs in the sorted form, in which the sorting is done on the key.

54. What is Singleton design pattern in C#?

Singleton design pattern in C# has just one instance that gives global access to it.

55. What is tuple in C#?

Tuple is a data structure to represent a data set that has multiple values that could be related to each other.

56. What are Events?

An event is a notice that something has occurred.

57. What is the Constructor Chaining in C#?

With Constructor Chaining, an overloaded constructor can be called from another constructor. The constructor must belong to the same class.

58. What is a multicasting delegate in C#?

Multicasting of delegates helps users to point to more than one method in a single call.

59. What are Accessibility Modifiers in C#?

Access Modifiers are terms that specify a program's member, class, or datatype's accessibility.

60. What is a Virtual Method in C#?

In the parent class, a virtual method is declared that can be overridden in the child class. We construct a virtual method in the base class using the virtual keyword, and that function is overridden in the derived class with the Override keyword.

61. What is Multithreading with .NET?

Multi-threading refers to the use of multiple threads within a single process. Each thread here performs a different function.

62. In C#, what is a Hash table class?

The Hash table class represents a collection of key/value pairs that are organized based on the hash code of the key.

63. What is LINQ in C#?

LINQ refers to Language Integrated Query. It provides .NET languages (like C#) the ability to generate queries to retrieve data from the data source.

64. Why can't a private virtual procedure in C# be overridden?

Private virtual methods are not accessible outside of the class.

65. What is File Handling in C#?

File handling includes operations such as creating the file, reading from the file, and appending the file, among others

66. Why are Async and Await used in C#?

Asynchronous programming processes execute independently of the primary or other processes. Asynchronous methods in C# are created using the Async and Await keywords.

67. What are I/O classes in C#?

In C#, the System.IO namespace contains multiple classes that are used to conduct different file operations such as creation, deletion, closure, and opening.

68. What is an Indexer in C#?

An indexer is a class property that allows you to access a member variable of another class using array characteristics.

69. What is Thread Pooling in C#?

In C#, a Thread Pool is a group of threads. These threads are used to do work without interfering with the principal thread's operation.

70.What exactly do you mean by regular expressions in C#?

A regular expression is a pattern that can be used to match a set of input. Constructs, character literals, and operators are all possible.

71. Can you write a code on Boxing and Unboxing in C#?

Ans: The methods of boxing and unboxing are used for type conversions. Boxing is converting from a value type to a reference type. Boxing is an implicit conversion. Here's an example of C# boxing.

```
// Boxing

int num1 = 1;

Object obj = num1;

Console.WriteLine(num1);

Console.WriteLine(obj);
```

Unboxing is the process of converting from a reference type to a value type. Here's a C# example of unboxing.

```
// Unboxing

Object obj2 = 1;

int num2 = (int) obj;
```

```
Console.WriteLine(num2);

Console.WriteLine(obj);
```

72. In C#, What is the difference between a struct and a class?

Ans: Class and struct are both user-defined data types, but they differ significantly:

Struct	Class
The struct is a value type in C# inherited from System.Value Type.	The class is a reference type in C# inherited from the System.Object Type.
Struct is usually used for smaller amounts of data.	Classes are usually used for large amounts of data.
Struct can’t be inherited from other types.	Classes can be inherited from other classes.
A structure can't be abstract.	A class can be an abstract type.
No need to create an object with a new keyword. Do not have permission to create any default constructor.	We can create a default constructor.

73. What exactly is cohesion?

Ans: In OOPS, we write code in modules. Each module is responsible for a specific task. Cohesion measures how closely a module's responsibilities are related.

Greater cohesion is always preferable. Benefits of increased cohesion include:

- Module maintenance is improved.
- Improve reusability.

74.Why are strings immutable in C#?

Ans: String values are immutable, which means they cannot be changed once they have been created. Any change to a string value creates a new string instance, resulting in inefficient memory use and extraneous garbage collection. The System is malleable. When string values change, the Text.StringBuilder class should be used.

75. What are the distinctions between Array and Array List in C#?

Ans: Arrays store values or elements of the same data type, whereas array lists store values of various data types.
Arrays will use a fixed length, whereas array lists will not.

76. What is the distinction between System.String and System.Text.StringBuilder?

Ans: Systems.Strings are unchangeable. When we change the value of a string variable, we allocate new memory to the new value and release the previous memory allocation. System.Text.StringBuilder was created with the concept of a mutable string in mind, allowing a variety of operations to be performed without the need for a separate memory location for the modified string.

77. What exactly is reflection in C#?

Ans: During runtime, C# reflection extracts metadata from data types.

To implement reflection in the .Net framework, simply use the System.Reflection namespace in your program to retrieve the type, which can be any of the following:

Assembly

ConstructorInfo

MemberInfo

ParameterInfo

FieldInfo

EventInfo

PropertyInfo

78. What is the purpose of 'Data Type Conversion' in C#?

Ans: The purpose of data type conversion is to avoid situations of runtime error during data type change or conversion.

79.What is the GAC, and where can I find it?

Ans: The Global Assembly Cache is abbreviated as the GAC. Shared assemblies are stored in the GAC, allowing applications to share assemblies rather than having them distributed with each application. Thanks to versioning, multiple assembly versions can exist in the GAC—applications can specify version numbers in the config file. To manage the GAC, use the gacutil command line tool.

80. What is a circular reference in C#?

Ans: This is a situation in which multiple resources depend on each other, resulting in a lock condition and unused resources.

81.What is Garbage collection in c#?
The garbage collector (GC) is in charge of memory allocation and release. The garbage collector is a memory manager that runs in the background. You don't need to know how to allocate and release memory or how to manage the lifetimes of the things that use it.

82. What is the distinction between the methods Finalize() and Dispose()?

Ans: When we want an object to release unmanaged resources, we call dispose(). Finalize(), on the other hand, serves the same purpose but does not guarantee garbage collection of an object.

83. What exactly are Anonymous Types in C#?
Ans: Anonymous types enable us to create new types without having to define them. This method defines read-only properties in a single object without explicitly defining each type. Type is generated by the compiler and is only available for the current block of code. The compiler also infers the type of properties.

85. Explain your test automation framework in C#.

Answer:
I have worked with a hybrid framework using **Selenium WebDriver with C#**, structured with the following components:

- **MSTest/NUnit** as the test runner.
- **Page Object Model (POM)** to maintain UI objects and actions.
- **SpecFlow** for BDD-style scenarios.
- **ExtentReports** for reporting.
- **Log4Net/NLog** for logging.
- **Selenium Grid** and **Docker** for distributed test execution.
- **CI/CD integration** with Azure DevOps.

86. What is the Page Object Model? How is it implemented in C#?

Answer:
POM is a design pattern that creates an object repository for web UI elements. It improves test maintenance and reduces code duplication.

Implementation in C#:

```
csharp
CopyEdit
public class LoginPage
{
    private IWebDriver driver;
    public LoginPage(IWebDriver driver) => this.driver = driver;

    private IWebElement Username => driver.FindElement(By.Id("username"));
    private IWebElement Password => driver.FindElement(By.Id("password"));
    private IWebElement LoginButton => driver.FindElement(By.Id("login"));

    public void Login(string user, string pass)
    {
        Username.SendKeys(user);
        Password.SendKeys(pass);
        LoginButton.Click();
    }
}
```

87. Difference between `Assert` and `Verify` in automation testing.

Answer:

- `Assert` stops the test execution on failure.
- `Verify` continues the execution even if verification fails.

In C#, assertions are provided by `NUnit.Assert` or `MSTest.Assert`

88. How do you handle dynamic elements in Selenium with C#?

Answer:

- Use dynamic XPath or CSS selectors.
- Wait until the element is stable using **WebDriverWait** and **ExpectedConditions**.
- Example:

```
WebDriverWait wait = new WebDriverWait(driver, TimeSpan.FromSeconds(10));
IWebElement element =
wait.Until(ExpectedConditions.ElementExists(By.XPath("//div[contains(@id,'u
ser_')]")));
```

88. How do you manage test data in your framework?

Answer:

- Using **JSON, XML, Excel** or **CSV** for external test data.
- For BDD scenarios, use SpecFlow tables.
- Example of reading JSON:

```
var jsonData = File.ReadAllText("TestData.json");
var data = JsonConvert.DeserializeObject<TestData>(jsonData);
```

89. What are waits in Selenium and which one do you use most?

Answer:

- **Implicit Wait**: Applied globally.
- **Explicit Wait**: Waits for specific condition.
- **Fluent Wait**: Similar to explicit but with polling intervals.

I prefer **Explicit Waits** for better control:

```
WebDriverWait wait = new WebDriverWait(driver, TimeSpan.FromSeconds(10));
wait.Until(ExpectedConditions.ElementToBeClickable(By.Id("submit")));
```

90. How do you execute tests in parallel using C#?

Answer:

- Use **NUnit Parallelizable** attribute:

```
 [TestFixture]
[Parallelizable(ParallelScope.All)]
public class LoginTests
```

- Configure thread count in the `.runsettings` file or via command line.

91. How do you integrate your tests with CI/CD pipelines?

Answer:

- Using **Azure DevOps** or **Jenkins**, I:
  - Push code to Git.
  - Trigger test runs using `.yml` pipelines.
  - Publish test results (e.g., using `dotnet test --logger:trx`) and attach reports.
  - Example for Azure pipeline step:

```
- script: dotnet test --logger:trx
```

92. How do you handle exceptions and logging in your framework?

Answer:

- Use `try-catch` blocks to catch exceptions.
- Log exceptions using **Log4Net** or **NLog**:

```
log.Error("Error in login", ex);
```

93. How do you perform API testing in C#?

Answer:
Using **RestSharp** or **HttpClient**:

```
var client = new RestClient("https://api.example.com");
var request = new RestRequest("users/1", Method.GET);
var response = client.Execute(request);
Assert.AreEqual(200, (int)response.StatusCode);
```

94. What are common challenges you've faced in automation and how did you resolve them?

Answer:

- **Flaky Tests** – Resolved by using dynamic waits and retry logic.
- **Element Not Found** – Improved locator strategy and DOM inspection.
- **Parallel Execution Issues** – Used thread-safe WebDriver instances and separate browser sessions.
- **Test Maintenance** – Used Page Factory/POM and centralized utility methods.

95. What is SpecFlow and how do you use it in your framework?

Answer:
SpecFlow is a BDD tool for .NET. It allows writing scenarios in Gherkin (Given-When-Then) syntax and binds them to step definitions in C#.

Example:

```
Scenario: Successful Login
Given I am on the login page
When I enter valid credentials
Then I should be redirected to the homepage
```

Step Definition:

```
 [Given(@"I am on the login page")]
public void GivenIAmOnTheLoginPage() {
    driver.Navigate().GoToUrl("http://example.com/login");
}
```

97. What design patterns have you used in test automation?

Answer:

- **Page Object Model (POM)** – For separating UI logic from test logic.
- **Singleton** – To maintain a single WebDriver instance.
- **Factory Pattern** – For creating WebDriver instances based on browser type.
- **Builder Pattern** – For complex test data creation.
- **Facade Pattern** – To simplify interactions across multiple services or page classes.

98. How do you manage cross-browser testing using Selenium in C#?

Answer:
I use a **factory pattern** to manage multiple browser drivers. Example:

```
public class WebDriverFactory
{
    public static IWebDriver GetDriver(string browser)
    {
        return browser.ToLower() switch
        {
            "chrome" => new ChromeDriver(),
            "firefox" => new FirefoxDriver(),
            "edge" => new EdgeDriver(),
            _ => throw new ArgumentException("Unsupported browser"),
        };
    }
```

```
}
driver = WebDriverFactory.GetDriver("chrome");
```

For scaling, I use **Selenium Grid**, **Docker**, or cloud services like **BrowserStack** or **LambdaTest**.

99. How do you generate and manage reports in your C# automation framework?

Answer:
I use **ExtentReports** to generate interactive HTML reports with screenshots, logs, and test status.

```
var htmlReporter = new ExtentHtmlReporter("extentReport.html");
var extent = new ExtentReports();
extent.AttachReporter(htmlReporter);
var test = extent.CreateTest("Login Test");
test.Pass("Login successful");
extent.Flush();
```

I attach these reports to CI/CD pipelines for stakeholders to view results.

100. How do you capture screenshots on test failure?

```
public void TakeScreenshot(string testName)
{
    ITakesScreenshot ts = (ITakesScreenshot)driver;
    Screenshot screenshot = ts.GetScreenshot();
    screenshot.SaveAsFile($"{testName}.png", ScreenshotImageFormat.Png);
}
```

I call this method in `TearDown` or `TestCleanup` methods when a test fails.

101. How do you manage browser sessions in parallel execution?

Answer:
By using **ThreadLocal** for WebDriver:

```
[ThreadStatic]
private static IWebDriver driver;

public IWebDriver GetDriver()
{
    if (driver == null)
    {
        driver = new ChromeDriver();
    }
    return driver;
}
```

This ensures each test thread has its own driver instance, avoiding conflicts.

102. How do you ensure test reusability and maintainability?

Answer:

- Use reusable **utility methods** (e.g., dropdown selectors, wait handlers).
- Apply **POM and separation of concerns**.
- Avoid hardcoding; use **configuration files** for URLs, credentials, etc.
- Maintain **test data separately**.
- Use **naming conventions and folder structure**.

103. How do you perform database validation in C# automation?

Answer:
Using **ADO.NET** or **Dapper**:

```
string connStr = "your_connection_string";
using(SqlConnection conn = new SqlConnection(connStr))
{
    conn.Open();
    SqlCommand cmd = new SqlCommand("SELECT * FROM Users WHERE ID=1",
conn);
    SqlDataReader reader = cmd.ExecuteReader();
    while(reader.Read())
    {
        Console.WriteLine(reader["Username"].ToString());
    }
}
```

I use this to validate backend data after UI actions.

104. How do you handle file upload/downloads in Selenium using C#?

Upload:

```
driver.FindElement(By.Id("upload")).SendKeys("C:\\path\\file.txt");
```

Download:

- Set browser preferences to auto-download files to a folder.
- Example (Chrome):

```
var options = new ChromeOptions();
```

```
options.AddUserProfilePreference("download.default_directory",
"C:\\Downloads");
```

Then verify file existence:

```
Assert.IsTrue(File.Exists("C:\\Downloads\\report.pdf"));
```

105. What are your best practices for writing automation test cases?

Answer:

- Keep test cases **atomic and independent**.
- Follow **AAA pattern** (Arrange, Act, Assert).
- Use **meaningful names** for test methods.
- Avoid test dependencies.
- Parameterize tests for data-driven execution.
- Always **clean up resources** in `TearDown` or `Dispose`.

106. What are some commonly used attributes in NUnit/MSTest?

NUnit:

- `[Test]`, `[SetUp]`, `[TearDown]`
- `[TestCase()]` – data-driven
- `[Parallelizable]`
- `[Ignore]`, `[Category]`

MSTest:

- `[TestMethod]`, `[TestInitialize]`, `[TestCleanup]`
- `[DataRow()]`, `[DataTestMethod]`
- `[TestCategory]`, `[ExpectedException]`

107. How do you implement logging in your framework?

Answer:
Using **Log4Net** or **NLog**:

log4net.config

```
xml
CopyEdit
<log4net>
  <appender name="FileAppender" type="log4net.Appender.FileAppender">
    <file value="Logs/log.txt" />
    <layout type="log4net.Layout.PatternLayout">
      <conversionPattern value="%date %-5level %message%newline" />
    </layout>
```

```
  </appender>
  <root>
    <level value="DEBUG" />
    <appender-ref ref="FileAppender" />
  </root>
</log4net>
```

Usage in code:

```
private static readonly ILog log = LogManager.GetLogger(typeof(ClassName));
log.Info("Starting test...");
```

108. What are common causes of flaky tests and how do you address them?

Causes:

- Timing issues.
- Dynamic elements or incorrect locators.
- Environmental instability (network, test data).
- Dependencies between tests.

Fixes:

- Use proper **waits** and **stable locators**.
- Isolate tests.
- Use retry logic or retry analyzers.
- Ensure **clean test environments**.

109. How would you test a React or Angular application using Selenium?

Answer:

- React/Angular components often update asynchronously, so I use:
    - `WebDriverWait` for element presence and visibility.
    - Use **browser dev tools** to inspect shadow DOM or components.
    - Avoid hardcoded waits; use **JavaScriptExecutor** if needed.

Example:

```
IJavaScriptExecutor js = (IJavaScriptExecutor)driver;
js.ExecuteScript("return document.readyState").Equals("complete");
```

:

🔷 C# – Classes, OOPs, Abstract Classes & Interfaces

✅ 1. What are the four pillars of Object-Oriented Programming (OOP) in C#?

Answer:

1. **Encapsulation** – Wrapping data and methods into a single unit (class).
2. **Abstraction** – Hiding internal implementation and showing only necessary details.
3. **Inheritance** – Acquiring properties and behavior of another class.
4. **Polymorphism** – Ability to take many forms (method overloading/overriding).

✅ 2. What is the difference between a Class and an Object in C#?

Answer:

- A **class** is a blueprint or template for creating objects.
- An **object** is an instance of a class.

```
public class Car {
    public string Color;
    public void Drive() { Console.WriteLine("Driving"); }
}

Car myCar = new Car(); // object
```

✅ 3. What is the difference between `public`, `private`, `protected`, and `internal` in C#?

Answer:

Access Modifier	Scope
`public`	Accessible from anywhere.
`private`	Accessible only within the same class.
`protected`	Accessible within the class and its derived classes.
`internal`	Accessible within the same assembly/project.

✅ 4. What is Inheritance? Give an example in C#.

Answer:
Inheritance is a mechanism to create a new class from an existing class.

```
public class Vehicle {
    public void Start() => Console.WriteLine("Vehicle starting...");
}

public class Car : Vehicle {
    public void Drive() => Console.WriteLine("Car driving...");
}
```

✅ 5. What is Polymorphism in C#? Explain Method Overloading vs Overriding.

Answer:

- **Overloading**: Same method name, different parameters (compile-time).

```
public void Print() {}
public void Print(string msg) {}
```

- **Overriding**: Redefining a base class method in the derived class (runtime).

```
public class Animal {
    public virtual void Sound() => Console.WriteLine("Generic Sound");
}

public class Dog : Animal {
    public override void Sound() => Console.WriteLine("Bark");
}
```

✅ 6. What is an Abstract Class in C#?

Answer:
An **abstract class** cannot be instantiated. It can have both abstract (no body) and non-abstract methods.

```
public abstract class Shape {
    public abstract double Area();
    public void Display() => Console.WriteLine("Displaying shape");
}

public class Circle : Shape {
    public override double Area() => Math.PI * 3 * 3;
}
```

✅ 7. What is an Interface in C#?

Answer:
An **interface** defines a contract with no implementation. Classes implement interfaces.

```
public interface IDriveable {
    void Drive();
}

public class Car : IDriveable {
    public void Drive() => Console.WriteLine("Driving...");
}
```

✅ 8. Difference between Abstract Class and Interface in C#

Feature	Abstract Class	Interface
Inheritance	Single inheritance only	Multiple interfaces
Members	Can have fields, constructors	Only methods/properties
Access modifiers	Allowed	Not allowed (everything is public)
Use case	Shared base implementation	Contract without implementation

---

✅ 9. Can a class implement multiple interfaces in C#? Can it inherit multiple classes?

Answer:

- **Yes**, a class can implement **multiple interfaces**:

```
public class Car : IDriveable, IFuelable {}
```

- **No**, C# does **not support multiple class inheritance**, but allows multiple interfaces.

✅ 10. What is `sealed` class in C#?

Answer:
A `sealed` class cannot be inherited.

```
public sealed class Logger {}
```

This is useful when you want to prevent further extension of a class (e.g., utility classes).

✅ 11. What is the difference between `virtual`, `override`, and `new` keywords?

Answer:

- `virtual`: Used in base class to allow overriding.
- `override`: Used in derived class to override a virtual method.
- `new`: Hides base class method without overriding.

```
public class Base {
    public virtual void Show() {}
}

public class Derived : Base {
    public override void Show() {} // overrides base
    public new void ShowAnother() {} // hides base
}
```

✅ 12. What are constructors and types in C#?

Answer:
Constructors initialize objects. Types:

- **Default constructor**
- **Parameterized constructor**
- **Static constructor** (initializes static members)
- **Copy constructor** (not built-in like C++, but can be implemented manually)

```
public class Person {
    public string Name;
    public Person(string name) => Name = name;
}
```

✅ 13. What is an Interface Segregation Principle (ISP)? How do you implement it in C#?

Answer:
ISP states that no client should be forced to depend on methods it doesn't use.

Bad:

```
public interface IAnimal {
    void Fly(); // not every animal flies
}
```

Good:

```
public interface IFlyable {
    void Fly();
}

public interface IRunnable {
    void Run();
}
```

✅ 14. What is Dependency Injection (DI)? Have you used it in test automation?

Answer:
DI is a design pattern where dependencies are injected rather than created inside a class.

In automation:

- Use **constructor injection** for WebDriver, utilities, etc.

```
public class LoginPage {
    private IWebDriver driver;
    public LoginPage(IWebDriver driver) => this.driver = driver;
}
```

Used with **SpecFlow**, **NUnit**, and **IoC containers** like Autofac or Microsoft.Extensions.DependencyInjection.

✅ 15. Can you instantiate an interface or abstract class in C#?

Answer:
No, you cannot instantiate them directly.

```
IShape shape = new Circle(); // ✅
Shape shape = new Circle();  // ✅
new IShape(); // ❌
new Shape();  // ❌
```

🔷 Can we create an object of an abstract class or interface in C#?

✅ Short Answer:

- **NO**, we **cannot create an object directly** from an **abstract class** or **interface**.
- But we **can reference** a **concrete (non-abstract) object** using an abstract class or interface **reference**.

🔹 1. Abstract Class:

❌ Not allowed:

```
csharp
CopyEdit
abstract class Animal {
    public abstract void Speak();
```

```
}

// Animal animal = new Animal(); ❌ Compilation error
```

✅ Allowed:

```
csharp
CopyEdit
class Dog : Animal {
    public override void Speak() => Console.WriteLine("Bark");
}

Animal animal = new Dog(); // ✅ Valid: reference is abstract, object is
concrete
animal.Speak();  // Output: Bark
```

🔹 2. Interface:

❌ Not allowed:

```
csharp
CopyEdit
interface IShape {
    void Draw();
}

// IShape shape = new IShape(); ❌ Compilation error
```

✅ Allowed:

```
csharp
CopyEdit
class Circle : IShape {
    public void Draw() => Console.WriteLine("Drawing Circle");
}

IShape shape = new Circle(); // ✅ Valid
shape.Draw(); // Output: Drawing Circle
```

🔷 Why can't we instantiate them?

- **Abstract class**: Incomplete – contains unimplemented (`abstract`) methods.
- **Interface**: Pure abstraction – contains only method/property signatures, no body.

So the **compiler restricts instantiation** because there's no complete implementation to instantiate.

◆ Real-world QA Automation Analogy:

If you have an interface like `IDriver`, you can’t "drive" an interface, but you can have:

```
public interface IDriver {
    void Start();
}

public class ChromeDriver : IDriver {
    public void Start() => Console.WriteLine("Chrome starts");
}

IDriver driver = new ChromeDriver(); // ✅
```

This is key in frameworks using **polymorphism** and **dependency injection**.

◆ 1. Can you explain how you would use an interface in your automation framework?

Sample Answer:
In my automation framework, I define interfaces to abstract actions or components. For example:

```
public interface ILoginPage {
    void Login(string username, string password);
}
```

Then I implement it:

```
public class LoginPage : ILoginPage {
    private IWebDriver driver;
    public LoginPage(IWebDriver driver) => this.driver = driver;

    public void Login(string username, string password) {
        driver.FindElement(By.Id("user")).SendKeys(username);
        driver.FindElement(By.Id("pass")).SendKeys(password);
        driver.FindElement(By.Id("login")).Click();
    }
}
```

This helps in **decoupling**, **mocking**, and **reusability**, especially in **BDD (SpecFlow)** or DI scenarios.

◆ 2. Write a code snippet that demonstrates abstraction using an abstract class.

```
public abstract class Browser {
    public abstract void Open();
    public void Close() => Console.WriteLine("Closing browser...");
}

public class Chrome : Browser {
    public override void Open() => Console.WriteLine("Opening Chrome...");
}
```

```
public class Firefox : Browser {
    public override void Open() => Console.WriteLine("Opening Firefox...");
}
```

Usage:

```
Browser browser = new Chrome();
browser.Open();  // Output: Opening Chrome
browser.Close(); // Output: Closing browser...
```

◈ 3. Given an interface `ITestLogger`, implement two types of loggers: FileLogger and ConsoleLogger.

```
public interface ITestLogger {
    void Log(string message);
}

public class FileLogger : ITestLogger {
    public void Log(string message) {
        File.AppendAllText("log.txt", message + "\n");
    }
}

public class ConsoleLogger : ITestLogger {
    public void Log(string message) {
        Console.WriteLine(message);
    }
}
```

Usage in tests:

```
ITestLogger logger = new FileLogger();
logger.Log("Test started");

logger = new ConsoleLogger();
logger.Log("Test completed");
```

◈ 4. What will happen if you don't override all methods of an interface in the implementing class?

Answer:

- The class will not compile unless it is declared **abstract**.

```
public interface IShape {
    void Draw();
    double Area();
}

public class Circle : IShape {
    public void Draw() => Console.WriteLine("Drawing...");
    // public double Area() { return 0; } // Required
}
```

This will give a **compile-time error** if `Area()` is not implemented.

◈ 5. How do abstract classes help in code reusability in automation frameworks?

Answer:

- Abstract classes allow defining **common functionality** (e.g., opening browser, navigation).
- Derived test classes can **inherit and reuse** the logic.

```
public abstract class TestBase {
    protected IWebDriver driver;

    public void LaunchBrowser() {
        driver = new ChromeDriver();
    }

    public abstract void RunTest();
}

public class LoginTest : TestBase {
    public override void RunTest() {
        LaunchBrowser();
        driver.Navigate().GoToUrl("https://example.com");
    }
}
```

This avoids duplicating common methods like `LaunchBrowser()`.

◈ 6. Interface vs Abstract Class in Real Framework Example?

Answer:

Use Case	Interface	Abstract Class
Page contracts (`ILoginPage`)	👍 Yes	👎 Not ideal
Common setup (TestBase class)	👎 Not possible (no code)	👍 Yes (with base methods)
Multiple behaviors (ILogger, IDriver)	👍 Yes	👎 Only single inheritance

---

◈ 7. Can you pass an interface as a parameter in a method?

Answer:
Yes. This is common in **dependency injection**.

```
public class TestRunner {
    private readonly ITestLogger logger;

    public TestRunner(ITestLogger logger) => this.logger = logger;

    public void Run() {
        logger.Log("Running test...");
    }
}
```

Usage:

```
TestRunner runner = new TestRunner(new ConsoleLogger());
runner.Run();
```

◈ 8. How is polymorphism achieved with interfaces in your framework?

Answer:
By using the **interface type** to refer to **different implementations**.

```
ILoginPage loginPage;

if (browser == "chrome")
    loginPage = new ChromeLoginPage(driver);
else
    loginPage = new FirefoxLoginPage(driver);

loginPage.Login("user", "pass"); // Single call, multiple behaviors
```

This enables **browser-specific logic** while keeping test code clean and consistent.

◈ 1. What is C#?

Answer:
C# (pronounced "C-Sharp") is an object-oriented, modern programming language developed by Microsoft as part of the .NET framework. It is used to develop desktop, web, and automation applications.

◈ 2. What are the main data types in C#?

Answer:

Data Type	Example	Description
`int`	`int x = 10;`	Integer numbers

Data Type	Example	Description
`double`	`double d = 10.5;`	Decimal numbers
`char`	`char c = 'A';`	Single character
`string`	`string s = "Hello";`	Sequence of characters
`bool`	`bool b = true;`	Boolean value (true/false)

◆ 3. What is the difference between `==` and `.Equals()` in C#?

Answer:

- `==` checks **value** for value types, and **reference** for reference types.
- `.Equals()` checks **value/content** for both value and reference types (can be overridden).

```
string a = "hello";
string b = "hello";
Console.WriteLine(a == b);        // True
Console.WriteLine(a.Equals(b));   // True
```

◆ 4. What is the difference between `const` and `readonly`?

Feature	`const`	`readonly`
When initialized	At compile time	At runtime (in constructor)
Modifiability	Cannot be changed	Can be set once at runtime
Example	`const int x = 10;`	`readonly int y;` in constructor

◆ 5. What is a `namespace` in C#?

Answer:
A `namespace` is used to organize code and avoid name conflicts.

```
namespace MyProject.Automation {
    class LoginTest {}
}
```

◆ 6. What is the difference between `ref` and `out` parameters?

Keyword	Use	Initialization Required?	Direction
`ref`	Pass value by reference	Yes	In/Out
`out`	Return multiple values	No	Out only

```
void GetValues(out int x) {
    x = 10;
}
```

◆ 7. What is an `array` in C#?

Answer:
An array is a fixed-size collection of elements of the same type.

```
int[] numbers = new int[3] { 10, 20, 30 };
Console.WriteLine(numbers[1]); // Output: 20
```

◆ 8. What is a `List<T>` in C#? How is it different from an array?

Answer:

- `List<T>` is a generic collection that can grow dynamically.
- Arrays have fixed size.

```
List<int> nums = new List<int>() { 1, 2, 3 };
nums.Add(4);
```

◆ 9. What is a method in C#?

Answer:
A method is a block of code that performs a task.

```
public int Add(int a, int b) {
    return a + b;
}
```

◆ 10. What is a `class` and `object`?

- **Class**: Blueprint of an object.
- **Object**: Instance of a class.

```
public class Car {
    public string color;
}

Car myCar = new Car();
myCar.color = "Red";
```

◈ 11. What is a constructor in C#?

Answer:
A constructor is a special method used to initialize objects.

```
public class Person {
    public string Name;
    public Person(string name) {
        Name = name;
    }
}
```

◈ 12. What is the difference between `static` and non-static members?

Static Member	Non-Static Member
Belongs to the class	Belongs to an instance
Accessed via class name	Accessed via object
Example: `Math.Abs()`	`car.StartEngine()`

◈ 13. What is exception handling? How do you handle it in C#?

Answer:
It's used to catch and handle runtime errors using `try-catch-finally`.

```
try {
    int x = 10 / 0;
} catch (DivideByZeroException e) {
    Console.WriteLine("Cannot divide by zero.");
} finally {
    Console.WriteLine("Cleanup code.");
}
```

◈ 14. What are properties in C#?

Answer:
They encapsulate fields and provide controlled access.

```
private int age;
public int Age {
    get { return age; }
    set { age = value; }
}
```

◈ 15. What is the difference between `value types` and `reference types`?

Value Type	Reference Type
Stored in stack	Stored in heap
Holds actual value	Holds reference to value
Examples: int, float	Examples: class, string, array

◈ 1. What is the difference between `Dispose()` and `Finalize()`?

Feature	`Dispose()`	`Finalize()`
Purpose	Release unmanaged resources manually	Used to clean up before GC destroys object
Triggered by	Called explicitly via `IDisposable`	Called by Garbage Collector
Control	Deterministic	Non-deterministic
Can suppress?	N/A	Yes, using `GC.SuppressFinalize()`

◆ Example using `Dispose()`:

```
public class FileManager : IDisposable {
    private FileStream fileStream;

    public FileManager(string path) {
        fileStream = new FileStream(path, FileMode.Open);
    }

    public void Dispose() {
        fileStream.Close();
        fileStream.Dispose();
    }
}
```

◆ Example using `Finalize()`:

```
class Sample {
    ~Sample() {
        // Finalizer
        Console.WriteLine("Object is being finalized.");
    }
}
```

◈ 2. What is Managed and Unmanaged Code?

Term	Explanation
Managed Code	Code that runs under the control of the .NET CLR (e.g., C#, VB.NET). CLR handles memory, type safety, security, GC.
Unmanaged Code	Code that runs outside .NET (e.g., C/C++). You must manually manage memory.

- Example of Unmanaged: COM objects, native DLLs
- In automation, unmanaged resources include files, streams, database handles.

◈ 3. What is Boxing and Unboxing in C#?

Boxing: Converting a **value type** (int, char) to **object type**.
Unboxing: Extracting the value type from the object.

◆ Example:

```
int num = 123;
object obj = num; // Boxing

int unboxed = (int)obj; // Unboxing
```

Note: Boxing adds performance overhead, so avoid it in loops or high-performance code.

◈ 4. What is Auto-Boxing?

Auto-boxing is an implicit conversion of a value type to object type.
In C#, this happens when a value type is assigned to an object or passed to a method expecting an object.

```
int x = 5;
object o = x; // Auto-boxing
```

◈ 5. What is a `sealed` class in C#?

Answer:
A sealed class **cannot be inherited**. Use it when you want to prevent further extension.

```
sealed class Logger {
    public void Log(string msg) => Console.WriteLine(msg);
}

// class MyLogger : Logger { } ✕ Compile-time error
```

Use in automation when you want to lock behavior (e.g., utility/helper classes).

◈ 6. What is a `partial` class in C#?

Answer:
Allows a class definition to be **split across multiple files**. Useful in auto-generated code (e.g., Windows Forms, ASP.NET).

```
// File1.cs
public partial class TestClass {
    public void Method1() {}
}

// File2.cs
public partial class TestClass {
    public void Method2() {}
}
```

Compiler merges them into a single class during compilation.

◈ 7. What is a `static` class in C#?

Answer:
A static class:

- Cannot be instantiated.
- Contains only static members.
- Often used for utility or helper methods.

```
public static class MathUtils {
    public static int Add(int x, int y) => x + y;
}

// Usage:
int result = MathUtils.Add(3, 4);
```

◈ 8. What is an abstract class and how is it different from an interface?

Feature	Abstract Class	Interface
Can have implementation	✓ Yes	✗ Until C# 8 (default impl allowed from C# 8)
Constructors	✓ Yes	✗ No
Fields	✓ Yes	✗ No

Feature	Abstract Class	Interface
Multiple inheritance	✗ No	✓ Yes
Use when	Partial abstraction	Total abstraction

◆ 9. What is a `readonly` field vs `const`?

Feature	`readonly`	`const`
Set when	Runtime	Compile-time
Modifier type	Instance-level	Static by default

Example

```
readonly int version; // set in constructor
const int PI = 3;     // must assign immediately
```

◆ 10. What is the difference between `public`, `private`, `protected`, and `internal`?

Modifier	Accessible from
`public`	Anywhere
`private`	Only in the same class
`protected`	In same class or derived class
`internal`	Within the same assembly (project)

✓ Bonus: Quick Interview One-Liners

- **Dispose()** → For releasing unmanaged resources **manually**.
- **Finalize()** → Called by GC before object destruction.
- **Boxing** → Value type → Object.
- **Unboxing** → Object → Value type.
- **Sealed** → Prevent inheritance.
- **Partial** → Split class across files.
- **Static** → No instance allowed, only static members.
- **Managed code** → Code under CLR.
- **Unmanaged code** → Native code, manual memory management.

.

◈ 1. Generics in C#

Q: What are generics? Why are they used?

Answer:
Generics allow you to create classes, methods, and collections that work with **any data type**, providing **type safety** and **performance**.

◆ Example: Generic Method

```
public class Utils {
    public void Print<T>(T value) {
        Console.WriteLine(value);
    }
}
```

◆ Generic List vs ArrayList

```
List<int> nums = new List<int>();     // Type-safe
ArrayList list = new ArrayList();     // Not type-safe
```

◈ 2. Delegates in C#

Q: What is a delegate?

Answer:
A delegate is a **type-safe function pointer**—it allows methods to be **passed as parameters**.

◆ Syntax:

```
public delegate void LogHandler(string message);

public class Logger {
    public void Log(string msg) {
        Console.WriteLine(msg);
    }
}
```

◆ Using Delegate:

```
LogHandler handler = new Logger().Log;
handler("Test passed");
```

◈ 3. Multicast Delegates

Q: What is a multicast delegate?

Answer:
A delegate that can **point to multiple methods**.

```
LogHandler handler = Method1;
handler += Method2;  // Adds another method
handler("Message");  // Both methods will run
```

◆ 4. Events in C#

Q: What is an event?

Answer:
An event is a way for a class to **notify other classes** when something happens. It's built on top of delegates.

◆ Example:

```
public class Button {
    public event EventHandler Clicked;

    public void Click() {
        Clicked?.Invoke(this, EventArgs.Empty);
    }
}
```

◆ Subscribing to event:

```
Button btn = new Button();
btn.Clicked += (s, e) => Console.WriteLine("Button clicked");
btn.Click();
```

Events are often used in **UI automation frameworks** or **custom logging mechanisms**.

◆ 5. LINQ (Language Integrated Query)

Q: What is LINQ in C#?

Answer:
LINQ allows you to **query collections** using SQL-like syntax in C#.

◆ Example: Filtering a list

```
List<int> nums = new List<int>() { 1, 2, 3, 4, 5 };
var evens = nums.Where(n => n % 2 == 0);

foreach (var num in evens)
    Console.WriteLine(num);
```

◈ 6. Useful LINQ Methods

Method	Purpose
`Where()`	Filter data
`Select()`	Transform data
`Any()`	Returns true if any element matches
`All()`	Returns true if all elements match
`First() / FirstOrDefault()`	Get first matching element
`OrderBy()`	Sorts data

◆ LINQ on objects:

```
var users = new List<User> {
    new User { Name = "Alice", Age = 25 },
    new User { Name = "Bob", Age = 30 }
};

var adults = users.Where(u => u.Age > 18);
```

◈ 7. Anonymous Methods & Lambda Expressions

◆ Anonymous method

```
delegate void Printer(string message);

Printer print = delegate(string msg) {
    Console.WriteLine(msg);
};
```

◆ Lambda Expression

```
Printer print = msg => Console.WriteLine(msg);
print("Hello");
```

These are heavily used in **LINQ** and **event handling**.

◈ 8. Func, Action, Predicate

Type	Description	Return Type
`Action<T>`	Method with input, no return	void

Type	Description	Return Type
`Func<T, TResult>`	Method with input and return	TResult
`Predicate<T>`	Method that returns bool	bool

◆ Examples:

```
Action<string> show = msg => Console.WriteLine(msg);
Func<int, int, int> add = (a, b) => a + b;
Predicate<int> isEven = n => n % 2 == 0;
```

✓ Summary Table

Concept	Use Case	In Automation
Generics	Type-safe code	Data handling, reusable methods
Delegates	Function pointers	Retry logic, logging
Events	Notify on action	UI triggers, loggers
LINQ	Query collections	Filter test cases, search logs
Lambda	Inline methods	Used in delegates/LINQ
Func/Action	Generic delegates	Callbacks, threading

C# programming Questions

1. **write a code to reverse a string**

Main.cs

Share

```
1  using System;
2
3  class Program
4  {
5      static void Main()
6      {
7          Console.Write("Enter a String: ");
8          string originalString = Console.ReadLine();
9          string reverseString = string.Empty;
10         for (int i = originalString.Length - 1; i >= 0; i--)
11         {
12             reverseString += originalString[i];
13         }
14         Console.WriteLine($"Reversed String: {reverseString}");
15     }
16 }
17
```

2.Write a code to find the maximum and minimum of array

Main.cs

Share Run

```
using System;
class Program
{
    static void Main()
    {
        // Assume the first element is the maximum
        int[] arr = { 14, 7, 25, 31, 10, 42 };
        int max = arr[0];
        for (int i = 1; i < arr.Length; ++i)
        {
            if (arr[i] > max)
            {
                max = arr[i];
            }
        }
        Console.WriteLine($"Maximum value in the array: {max}");
    }
}
```

Main.cs

Share Run

```
using System;
class Program
{
    static void Main()
    {
        // Assume the first element is the maximum
        int[] arr = { 14, 7, 25, 31, 10, 42 };
        int min = arr[0];
        for (int i = 1; i < arr.Length; ++i)
        {
            if (arr[i] < min)
            {
                min = arr[i];
            }
        }
        Console.WriteLine($"Maximum value in the array: {min}");
    }
}
```

3.write a code to find the Fibonacci series

Main.cs

Share Run

```
using System;
class Program
{
    static void Main()
    {
        int firstNumber = 0, secondNumber = 1, nextNumber, numberOfElements;
        Console.Write("Enter the number of elements to print: ");
        numberOfElements = int.Parse(Console.ReadLine());

        if (numberOfElements < 2)
        {
            Console.Write("Please enter a number greater than two.");
        }
        else
        {
            Console.Write(firstNumber + " " + secondNumber + " ");
            for (int i = 2; i < numberOfElements; i++)
            {
                nextNumber = firstNumber + secondNumber;
                Console.Write(nextNumber + " ");
                firstNumber = secondNumber;
                secondNumber = nextNumber;
            }
        }

        Console.ReadKey();
    }
}

```

4.write a code to find the factorial of a number

Main.cs

Share

```
using System;
class Program
{
    static void Main()
    {
        Console.Write("Enter a Number: ");
        int number = int.Parse(Console.ReadLine());

        long factorial = 1;
        for (int i = 1; i <= number; i++)
        {
            factorial *= i;
        }

        Console.WriteLine($"Factorial of {number} is: {factorial}");
    }
}

```

5.write a code to find a anagrams

Main.cs

```
using System;
using System.Linq;
class Program
{
    public static bool AreAnagrams(string str1, string str2)
    {
        // Remove non-alphanumeric characters and convert to lowercase
        str1 = new string(str1.Where(char.IsLetterOrDigit).ToArray()).ToLower();
        str2 = new string(str2.Where(char.IsLetterOrDigit).ToArray()).ToLower();
        // Sort the characters
        char[] sortedStr1 = str1.ToCharArray();
        char[] sortedStr2 = str2.ToCharArray();
        Array.Sort(sortedStr1);
        Array.Sort(sortedStr2);
        // Compare the sorted strings
        return sortedStr1.SequenceEqual(sortedStr2);
    }

    static void Main()
    {
        string input1 = "listen";
        string input2 = "silent";

        if (AreAnagrams(input1, input2))
        {
            Console.WriteLine($"{input1} and {input2} are anagrams!");
        }
        else
        {
            Console.WriteLine($"{input1} and {input2} are not anagrams.");
        }
    }
}

```

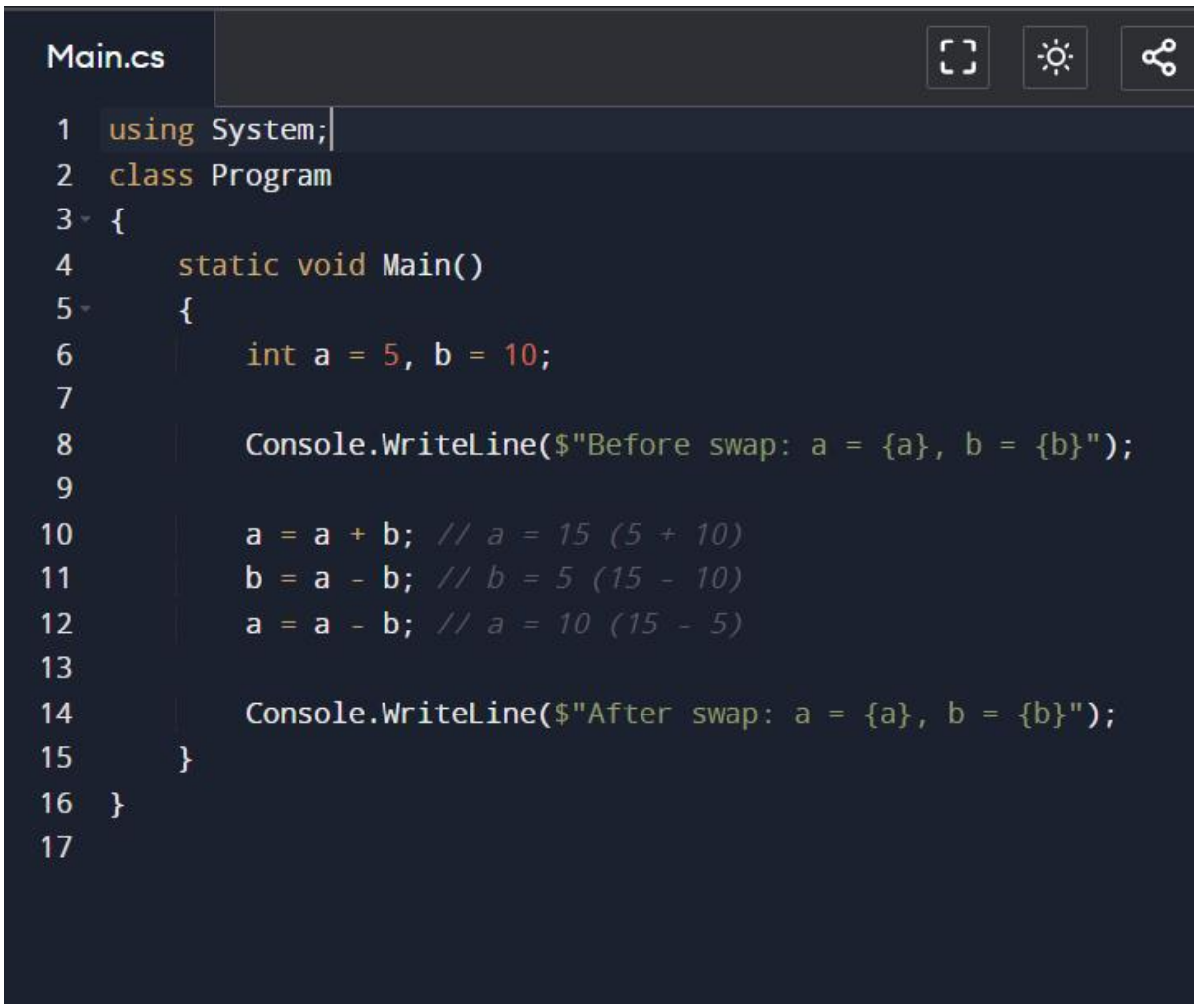
6. write a code to print star pattern in c#

Main.cs

```
using System;

class Program
{
    static void Main(string[] args)
    {
        string series = string.Empty;
        for (int i = 0; i <= 5; i++)
        {
            series += "*";
            Console.WriteLine(series);
        }

        Console.ReadLine();
    }
}

```

7.write a code to swap two numbers

Main.cs



```cs
using System;
class Program
{
    static void Main()
    {
        int a = 5, b = 10;

        Console.WriteLine($"Before swap: a = {a}, b = {b}");

        a = a + b; // a = 15 (5 + 10)
        b = a - b; // b = 5 (15 - 10)
        a = a - b; // a = 10 (15 - 5)

        Console.WriteLine($"After swap: a = {a}, b = {b}");
    }
}

```

8.write a code to find palindrome

Main.cs

```cs
using System;
class Program
{
    static void Main()
    {
        Console.Write("Enter a number: ");
        int num = int.Parse(Console.ReadLine());
        string original = num.ToString();
        char[] reversed = original.ToCharArray();
        Array.Reverse(reversed);
        string reversedString = new string(reversed);
        if (original == reversedString)
        {

            Console.WriteLine($"{num} is a palindrome!");
        }
        else
        {
            Console.WriteLine($"{num} is not a palindrome.");
        }
    }
}
```